HW 4 P#1

Minimizer $f(x^*) \le f(x)$ for every x in $[0,1]$

1. i) Algorithm: just return $\hat{x} = \frac{1}{2}$.

Runtime: $O(1)$ b/c the alg just returns a constant which takes $O(1)$ time b/c it doesn't depend on anything else

Proof: Let x^* be a minimizer of f on $[0,1]$. Both $\hat{x} = 1/2$ and x^* are in $[0,1]$, so $|\hat{x} - x^*| \le 1/2$. B/c f is L-Lipschitz, $f(\hat{x}) - f(x^*) \le L \cdot |\hat{x} - x^*| \le L/2 \le \epsilon$

ii) Algorithm:

```
1  a=0, b=1
2  while (b-a > 2ε/L){
3      M1 = a + (b-a)/3
4      M2 = b - (b-a)/3
5      if (f(M1) < f(M2)){
6          b = M2
7      }
8      else{
9          a = M1
10     }
11 }
12 return x̂ = (a+b)/2
```

HW4 Pt2

Runtime: Each iteration shrinks the interval to 2/3 of its length, i.e. after t iterations, $b-a=(2/3)^t$. The loop stops once $(2/3)^t \leq 2\varepsilon/L$, which takes at most $\log_{3/2}(L/2\varepsilon) = O(\log(L/\varepsilon))$ iterations. Each iteration makes 2 queries and does $O(1)$ work. ∴ the total is $O(\log(L/\varepsilon))$ queries and $O(\log(L/\varepsilon))$ time

Correctness: Let x^* be a minimizer of f on $[0,1]$. b/c f is strictly convex, f is strictly decreasing on $[0,x^*]$ and strictly increasing on $[x^*,1]$.

Claim: $x^* \in [a,b]$ at the start of every iteration. Initially $[a,b]=[0,1]$. Now assume $x^* \in [a,b]$.

- If $f(m_1) < f(m_2)$, then $x^* \leq m_2$. Otherwise $m_1 < m_2 < x^*$ lie where f is decreasing, which would give $f(m_1) > f(m_2)$. ∴, $x^* \in [a, m_2]$.

Now if $f(m_1) \geq f(m_2)$, then $x^* \geq m_1$, otherwise $x^* < m_1 < m_2$ lie where f is increasing, which would give $f(m_1) < f(m_2)$. ∴ $x^* \in [m_1, b]$. When the loop exits $b-a \leq 2\varepsilon/L$ and $x^* \in [a,b]$, so $|\hat{x}-x^*| \leq \varepsilon/L$. since f is L-Lipschitz

HW4 Pt3

2. i) Algorithm: just return $c_x = (1/n) \cdot \sum_i y_i$, the mean of the y-values.

Runtime: $O(n)$ b/c we do a passover n numbers to do sum then divide

Correctness: A 1-PC is the constant c_x on all of $[x_1, x_n]$ so the objective is $g(c) = \sum_i (c - y_i)^2$. Then $g'(c) = 2\sum_i (c - y_i) = 2(nc - \sum_i y_i)$, which is 0 exactly when $c = (1/n)\sum_i y_i$. B/c $g''(c) = 2n > 0$, g is convex, i.e. this critical point is a global minimizer.

ii) Algorithm: For a group of points $i \ldots j$, let $avg(i,j) = (y_i + \ldots + y_j)/(j-i+1)$ and let $cost(i,j) = \sum_{t=i}^{j} (y_t - avg(i,j))^2$

Subproblems: $DP[j]$ = Min cost (squared error + $C \cdot$ #Pieces) of fitting points $1 \ldots j$ w/ $DP[0] = 0$.

HW4 pt4

... recurrence: Min over $1 \le i \le j$ of $DP[i-1] + \text{cost}(i,j) + c$, where i is where the last group starts. Compute $\text{cost}(i,j)$ directly when considering each option, and store the minimizing i as $\text{start}[j]$.

Order: $j = 1, 2, \ldots, n$. $DP[j]$ only uses $DP[i-1]$ w/ $i-1 < j$

Output: Follow $\text{start}[]$ back from n to recover the groups. For a group $i \ldots j$, output the constant $\text{avg}(i,j)$ and the interval $[x_i, x_{j+1})$. Make the last interval $[x_i, x_n]$ instead.

Runtime: n DP entries and each takes a min over at most n options and each option computes $\text{cost}(i,j)$ in $O(n)$. So $n \cdot n \cdot O(n) = O(n^3)$. Brute-forcing = $O(n)$. $\therefore O(n^3)$

HW 4 Pt 5

correctness: the intervals are non-overlapping and ordered i.e. each covers a contiguous run of points. An interval containing no points can be merged into a neighbor, which keeps the order the same and saves k. So we can assume the k pieces split points $1, \dots, n$ into contiguous, non-empty groups.

We show by induction on j that $DP[j]$ is the optimal cost for points $1, \dots, j$. The base case $DP[0] = 0$ holds bc no points means no pieces. For $j \geq 1$, any valid split of points $1, \dots, j$ has a last group, say $i, \dots, j$. The rest is a split of points $1, \dots, i-1$, which costs at least $DP[i-1]$ by induction. By part (i) of Problem 2, the last group costs at least $cost(i,j) + c$. So $OPT(j) \geq \min_i [DP[i-1] + cost(i,j) + c] = DP[j]$. Every option in the min is achievable, so $DP[j] \leq OPT(j)$, which gives equality. $\therefore DP[n]$ is the minimum OP (2) and backtracking recovers the i that achieves it.

HW 4 P6

3. a) Symmetric A is positive semi-definite (PSD) if $x^T Ax \geq 0$ for all $x \in \mathbb{R}^d$.

i) If every $A_{S \times S}$ is PSD, then A is PSD. Take $S = [d]$. Then $A_{S \times S} = A$, which is PSD by assumption.

If A is PSD, then every $A_{S \times S}$ is PSD. Fix any $S \subseteq [d]$ and any $z \in \mathbb{R}^{|S|}$. Define $x \in \mathbb{R}^d$ by placing the entries of z at the indices in S and 0 everywhere else. Every term with $i \notin S$ or $j \notin S$ has a zero factor, so

$$x^T Ax = \sum_{i,j} x_i A_{ij} x_j = \sum_{i,j \in S} x_i A_{ij} x_j = z^T A_{S \times S} z$$

Since A is PSD, $x^T Ax \geq 0$, so $z^T A_{S \times S} z \geq 0$. $A_{S \times S}$ is also symmetric b/c A is. So $A_{S \times S}$ is PSD.

ii) If $A = B^T B$, then A is PSD. For any x, $x^T Ax = x^T B^T Bx = (Bx)^T (Bx) = \|Bx\|^2 \geq 0$.

If A is PSD, then $A = B^T B$ for some B. Since A is symmetric, the spectral theorem gives $A = U \Lambda U^T$, where U is orthogonal ($U^T U = I$) with eigenvectors $u_1, \ldots, u_d$ as its columns, and Λ is diagonal with the eigenvalues $\lambda_1, \ldots, \lambda_d$.

HW4 P7

... Each eigenvalue is non-negative;
$U_i^T A U_i = \lambda_i U_i^T U_i = \lambda_i$, and this is ≥ 0 b/c A is PSD. So we can let $\Lambda^{1/2}$ be the diagonal matrix w/ entries $\sqrt{\lambda_i}$ and set $B = \Lambda^{1/2} U^T$. Then $B^T B = U \Lambda^{1/2} \Lambda^{1/2} U^T = U \Lambda U^T = A$.

4. b) The gradient of $\frac{1}{2} x^T M x$ is $\frac{1}{2}(M + M^T) x = Mx$, b/c M is symmetric. The gradient of $v^T x$ is v. So $\nabla f(x) = Mx - v$.

Why $x^* = M^{-1} v$ is the global minimizer: M is positive definite, so all of its eigenvalues are positive and M is invertible, which means x^* is well-defined. Since $v = Mx^*$, we can re-write $f(x) = \frac{1}{2} x^T M x - x^{*T} M x$. Expanding gives $f(x) - f(x^*) = \frac{1}{2}(x - x^*)^T M (x - x^*)$. Since M is positive definite, the right side is ≥ 0 for every x, and equals 0 only when $x = x^*$, so x^* is the unique global minimizer. Equivalently, $\nabla f(x^*) = M M^{-1} v - v = 0$, and f is strictly convex b/c its Hessian M is positive definite.

HW4 P4

ii) Since $v = Mx^*$, we have $\nabla f(x) = Mx - Mx^* = M(x - x^*)$. Plugging this into the update (3):

$x' - x^* = (x - x^*) - (1/\lambda_1) M(x - x^*)$.

Sub $x - x^* = \sum_i c_i v_i$ and use $M v_i = \lambda_i v_i$:

$x' - x^* = \sum_i c_i v_i - (1/\lambda_1) \sum_i \lambda_i c_i v_i$

$= \sum_i (1 - \lambda_i/\lambda_1) c_i v_i$.

The v_i form an orthogonal basis, so the coefficients of $x' - x^*$ in that basis are $c_i' = (1 - \lambda_i/\lambda_1) c_i$ for all i.

iii) Since V is orthogonal ($V^T V = I$),

$\|x - x^*\|^2 = \|Vc\|^2 = c^T V^T V c = \sum_i c_i^2$.

In that same basis, by Parseval (ii),

$\|x' - x^*\|^2 = \sum_i (1 - \lambda_i/\lambda_1)^2 c_i^2$.

Since $\lambda_d \le \lambda_i \le \lambda_1$ for every i, we get

$0 \le 1 - \lambda_i/\lambda_1 \le 1 - \lambda_d/\lambda_1 = 1 - 1/k$. So

$\|x' - x^*\|^2 \le (1 - 1/k)^2 \sum_i c_i^2 = (1 - 1/k)^2 \|x - x^*\|^2$.

Both sides are non-negative so taking square roots gives $\|x' - x^*\| \le (1 - 1/k)\|x - x^*\|$.

Problem 5 link:
https://colab.research.google.com/drive/1pIrV_2JUKQfTfgTxZrAPcEhTsts8ga3-?usp=sharing